Algorytmy geometryczne - sprawozdanie 2

Otoczka wypukła

Adrianna Grobel
Data: 04.11.2025

1. Dane techniczne

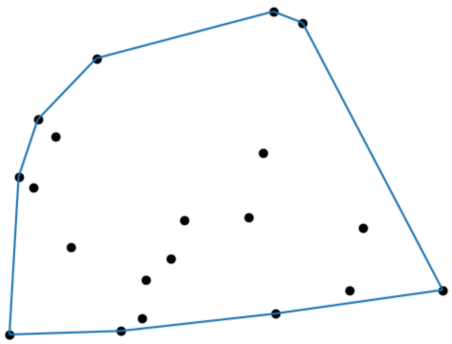
- System operacyjny - macOS
- Procesor - Apple M3
- Pamięć RAM - 16 GB
- Środowisko - Jupyter Notebook
- Język - Python 3.13.5
- Wykorzystane biblioteki - Numpy, Pandas, Matplotlib

2. Wstęp teoretyczny

2.1. Otoczka wypukła

Otoczka wypukła zbioru Q to najmniejszy zbiór wypukły zawierający Q. Dla dowolnej pary punktów $p, q \in \mathrm{CH}(Q)$ odcinek pq również leży w $\mathrm{CH}(Q)$.

Otoczka wypukła punktów na płaszczyźnie jest najmniejszym wielokątem wypukłym zawierającym zbiór Q. Według konwencji listę punktów otoczki z Q podaje się w porządku przeciwnym do ruchu wskazówek zegara. W przypadku punktów współliniowych, do otoczki należy jedynie ten najbardziej oddalony od poprzedniego punktu otoczki.
Rysunek 1 przedstawia przykładową otoczkę wypukłą (kolor niebieski), zdefiniowaną nad zadanym zbiorem punktów (kolor czarny).



Rysunek 1: Przkładowa otoczka wypukła

2.2. Algorytm Grahama

Algorytm Grahama to metoda znajdowania otoczki wypukłej zbioru punktów na płaszczyźnie opierająca się na kątowym sortowaniu punktów. Implementacja algorytmu przebiega w kilku etapach:

1. **Wybór punktu startowego:** Najpierw identyfikowany jest punkt P_0 o najmniejszej współrzędnej y. W przypadku, gdy istnieje kilka takich punktów, wybierany jest ten o najmniejszej współrzędnej x.

2. **Sortowanie punktów względem kąta:** Pozostałe punkty są sortowane rosnąco według kąta, jaki tworzą z punktem P_0 względem osi OX. Do określenia relacji

między punktami użyto wbudowanej funkcji sortującej, funkcji cmp_to_key() i funkcji compare(b, c), która za pomocą wyznacznika:

$\det t(P_0, b, c) = (P_{0x} - b_x) \cdot (c_y - b_y) - (P_{0y} - b_y) \cdot (c_x - b_x)$ sprawdza położenie trzech punktów:
- jeśli det > 0, punkt b zostaje przed c,
- jeśli det < 0, punkt c trafia przed b

Punkty leżące na tej samej prostej (tworzące ten sam kąt) są redukowane do pojedynczego punktu jeszcze przed zwróceniem posortowanej listy — zachowywany jest jedynie punkt najbardziej oddalony od P_0.

3. **Budowa otoczki:** Tworzony jest stos, który będzie przechowywał kandydatów na punkty otoczki. Po kolei sprawdzany jest każdy punkt z posortowanej listy:
   - jeśli dwa ostatnie punkty ze stosu wraz z rozpatrywanym punktem tworzą skręt w lewo, punkt dodawany jest na szczyt stosu,
   - w przypadku skrętu w prawo lub kolinearności, punkt ze szczytu stosu jest usuwany, a sprawdzanie powtarzane, aż zostanie zachowana poprawna kolejność.
4. **Wynik końcowy:** Po przejściu przez wszystkie punkty stos wraz z punktem P_0 zawiera kolejne wierzchołki otoczki wypukłej.

Złożoność obliczeniowa algorytmu wynosi $O(n \log n)$, natomiast złożoność pamięciowa to $O(n)$, gdzie n oznacza liczbę punktów w zbiorze.

2.3. Algorytm Jarvisa

Algorytm Jarvisa, znany również jako algorytm owijania prezentu, działa na zasadzie podejmowania lokalnie optymalnych decyzji. Zaimplementowany algorytm składa się z kilku kroków:

1. **Ustalenie punktu startowy:** Najpierw wyszukiwany jest punkt P_0 o najmniejszej współrzędnej y (w przypadku tych samych współrzędnych wybierany jest punkt o najmniejszej współrzędnej x). Punkt ten stanowi początek otoczki.
2. **Wybór kolejnego punktu otoczki:** W każdej iteracji wybieramy dowolny inny punkt ze zbioru, w tym przypadku next = (current + 1) % n (nie ma to większego znaczenia, jest wygodne, bo zapewnia, że bierzemy inny punkt niż current i jednocześnie nie wyjdziemy z tablicy). Punkt ten jest kandydatem do zostania kolejnym punktem otoczki. Następnie przechodząc przez wszystkie pozostałe punkty szukamy punktu, tworzącego najmniejszy skręt w lewo względem bieżącego punktu na otoczce.
   - W tym celu używana jest funkcja `compare`, bazująca na wyznaczniku `mat_det`, który określa orientację trzech punktów. Jeśli nowy punkt okazuje się lepszy od kandydata, zmieniamy punkt odniesienia.
   - W przypadku punktów współliniowych wybierany jest punkt najbardziej oddalony od bieżącego punktu, co zapewnia poprawne rozszerzenie otoczki. Odległość mierzona jest przy użyciu metryki Manhattan.

3. **Tworzenie otoczki:** Punkt wybrany jako kolejny jest dodawany do listy wynikowej. Proces powtarza się, aż algorytm wróci do punktu startowego, co oznacza zakończenie budowy otoczki.

4. **Wynik:** Funkcja zwraca listę punktów tworzących otoczkę wypukłą, od P_0 w kolejności odwrotnej do wskazówek zegara.

Teoretyczna złożoność czasowa algorytmu wynosi $O(\mathrm{nk})$, gdzie n jest liczbą punktów w zbiorze, a k - liczbą punktów należących do otoczki.

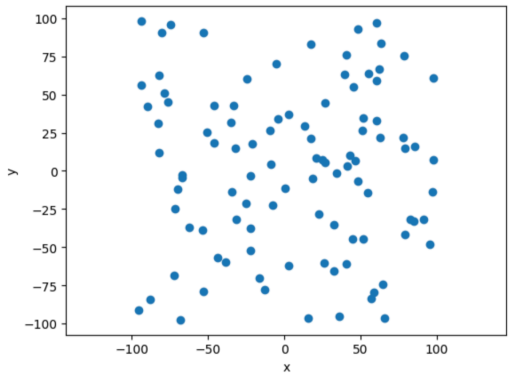
3. Realizacja ćwiczenia

3.1. Informacje wstępne

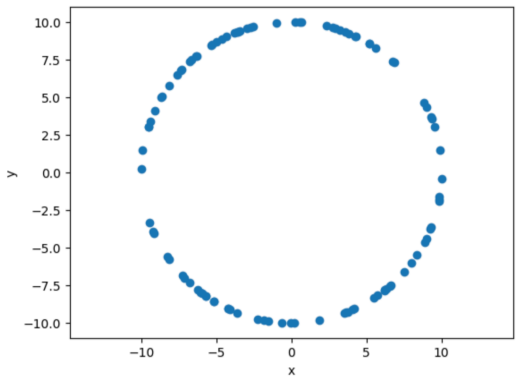
Celem ćwiczenia była implementacja algorytmów Jarvisa i Grahama, służących do wyznaczania otoczki wypukłej zbioru punktów na płaszczyźnie. Przeprowadzono również wizualizację działania obu algorytmów oraz analizę uzyskanych wyników, której celem było porównanie ich efektywności i skuteczności dla różnych układów punktów.

Na potrzeby realizacji zadania zostały wygenerowane następujące zestawy punktów o losowym, równomiernym rozkładzie:

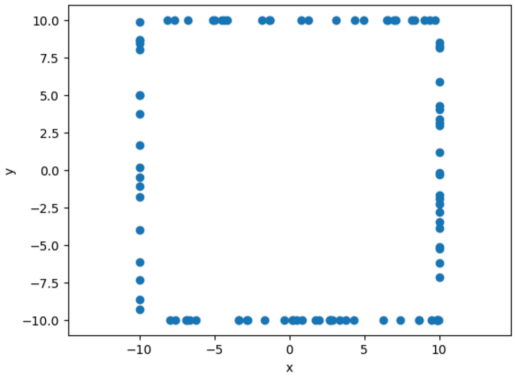
1. Zestaw A - 100 punktów na płaszczyźnie $[-100, 100]^2$
2. Zestaw B - 100 punktów leżących na okręgu w układzie współrzędnych o środku $O = (0, 0)$ i promieniu $R = 10$
3. Zestaw C - 100 punktów leżących na obwodzie prostokąta o wierzchołkach a, b, c, d o współrzędnych $a = (-10, -10)$, $b = (10, -10)$, $c = (10, 10)$, $d = (-10, 10)$
4. Zestaw D - zbiór zawierający wierzchołki kwadratu (0, 0), (10, 0), (10, 10), (0, 10) oraz punkty wygenerowane losowo w sposób następujący: po 25 punktów na dwóch bokach kwadratu leżących na osiach i po 20 punktów na przekątnych kwadratu dalej zwany zbiorem D
Rysunki Rysunek 2, Rysunek 3, Rysunek 4, Rysunek 5 przedstawiają wizualizację poszczególnych zbiorów.



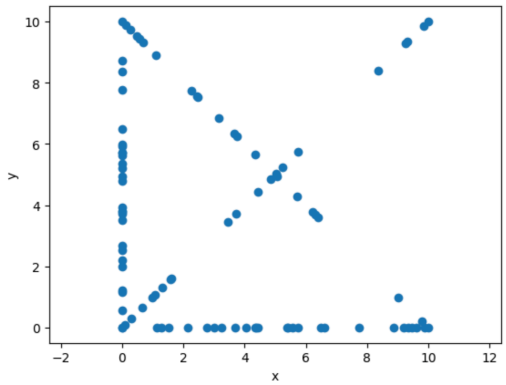
Rysunek 2: Zbiór A



Rysunek 3: Zbiór B



Rysunek 4: Zbiór C



Rysunek 5: Zbiór D

3.2. Generowanie punktów

Równomierny rozkład punktów został uzyskany z wykorzystaniem funkcji random.uniform z biblioteki numpy, generującej liczby zmiennoprzecinkowe typu float64 w określonym zakresie domkniętym. Aby zapewnić powtarzalność rezultatów, zastosowano stałą wartość `numpy.random.seed`.

Punkty należące do okręgu w Zestawie B wygenerowano przy użyciu funkcji trygonometrycznych, korzystając z parametrycznego równania okręgu o promieniu R i środku w punkcie O:

$$y = R \sin t + O_y$$
$$x = R \cos t + O_x$$

gdzie `t` to parametr rzeczywisty określający położenie punktu na okręgu.

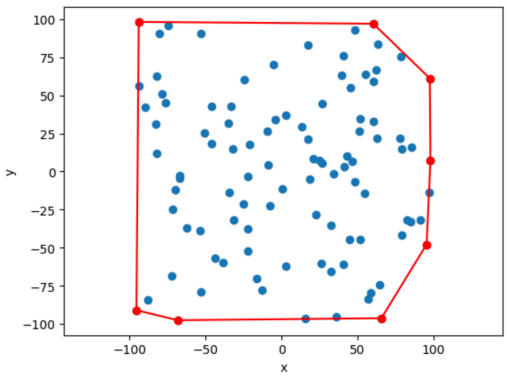
Jednostajny rozkład punktów w Zestawie C został uzyskany poprzez losowe generowanie punktów na obwodzie prostokąta o zadanych wierzchołkach A,B,C,D. W tym celu w każdej iteracji losowano jeden z czterech boków, a następnie wybierano współrzędne punktu z odpowiedniego zakresu przy użyciu funkcji numpy.random.uniform, zapewniającej równomierny rozkład wartości rzeczywistych. Dla boków poziomych (AB i CD) losowano współrzędną x, współrzędna y pozostawała stała. Analogicznie, dla boków pionowych (BC i DA) losowano współrzędną y, przy zachowaniu stałej wartości x.

Współrzędne punktów na bokach kwadratu w Zestawie D zostały wyznaczone w sposób analogiczny do punktów Zestawu C. Punkty na przekątnych kwadratu rozmieszczane są równomiernie wzdłuż prostych opisanych równaniami:
AC: $y = x - a_x + a_y$
BD: $y = -x + b_x + b_y$
x są losowane z przedziałów odpowiadających końcom przekątnych, po czym obliczana jest odpowiadająca im współrzędna y.

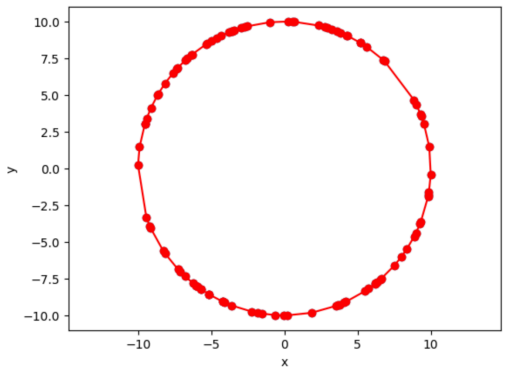
4. Wyniki działania algorytmów

4.1 Wizualizacja otoczek wypukłych

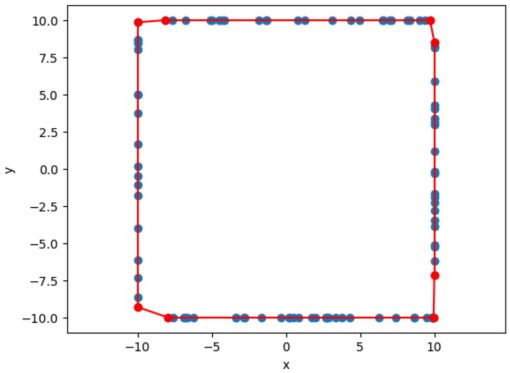
Rysunki Rysunek 6, Rysunek 7, Rysunek 8, Rysunek 9 przedstawiają otoczki wypukłe utworzone dla zestawów kolejno A, B, C, D wygenerowane algorytmem Grahama. Algorytm Jarvisa generował identyczne wykresy. Dla każdego punktu na wykresie kolor **niebieski** oznacza, że punkt należący do zbioru nie jest częścią otoczki wypukłej. Kolor **czerwony** zaznacza punkty należące do otoczki. W wygenerowanych animacjach działania algorytmów, załączonych razem ze sprawozdaniem, oznaczenia zostają te same, ale uzupełnione w algorytmie Grahama o kolor **czarny** - niesprawdzone jeszcze punkty i **szary** - punkty współliniowe niebrane pod uwagę, nienależące do otoczki. W algorytmie Jarvisa kolorem **pomarańczowym** oznaczane są punkty aktualnie sprawdzane.



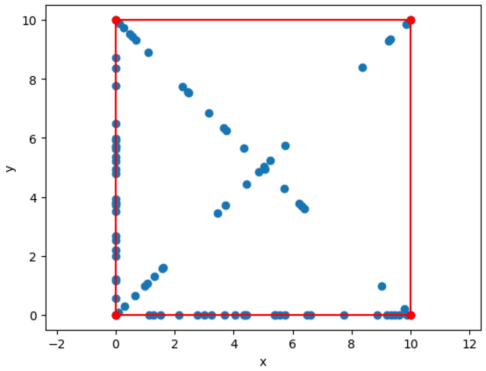
Rysunek 6: Otoczka wypukła zbioru A



Rysunek 7: Otoczka wypukła zbioru B



Rysunek 8: Otoczka wypukła zbioru C



Rysunek 9: Otoczka wypukła zbioru D

4.2. Analiza wyników działania algorytmów

Wyniki działania algorytmów przedstawione zostały w tabeli Tabela 1. Dla każdego zbioru testowego oba algorytmy zwracały taki sam zbiór punktów otoczki.
Możemy zauważyć, że otoczka zbioru A jest wyznaczona poprawnie (Rysunek 6), wielokąt wypukły zawiera wszystkie punkty zbioru.
W przypadku zbioru B (Rysunek 7) każdy punkt okręgu należy do otoczki.
Z kolei dla zbiorów C i D (Rysunek 8 i Rysunek 9) użyte implementacje skutecznie

poradziły sobie z punktami współliniowymi: otoczka zbioru C obejmuje osiem punktów na bokach prostokąta, natomiast dla zbioru D otoczka składa się z czterech wierzchołków kwadratu

Zestaw	Algorytm Grahama	Algorytm Jarvisa
A	13	13
B	100	100
C	8	8
D	4	4

Tabela 1: Liczba punktów należących do otoczki wyznaczonej przez konkretny algorytm

5. Porównanie działania algorytmów

5.1 Modyfikacja zbiorów

W celu przeprowadzenia testów czasowych algorytmów Jarvisa i Grahama dokonano modyfikacji generowanych zbiorów punktów opisanych w punkcie 3.2.
Do testów utworzono nowe zbiory danych o licznościach:

A, C, D: [100, 1000, 5000, 10000, 50000], B: [10, 100, 500, 1000, 5000].

Każdy zbiór został wygenerowany przy użyciu tych samych funkcji, jednak z innymi parametrami obszaru:
Zbiór A zawiera punkty rozmieszczone równomiernie w zakresie współrzędnych $[-1000, 1000]$,
Zbiór B składa się z punktów leżących na okręgu o środku w punkcie $(100, 100)$ i promieniu $R = 1000$,
Zbiór C obejmuje punkty rozmieszczone losowo na bokach prostokąta o wierzchołkach $(-500, 0)$, $(-500, 100)$, $(500, 100)$ oraz $(500, 0)$,
Zbiór D zawiera wierzchołki kwadratu $(0, 0)$, $(100, 0)$, $(100, 100)$ oraz $(0, 100)$ oraz dodatkowe punkty rozmieszczone w sposób zróżnicowany — na dwóch bokach przylegających do osi układu współrzędnych rozmieszczono po $0,4n - 1$ punktów i na każdej przekątnej po $0,1n - 1$ punktów, gdzie n oznacza całkowitą liczbę punktów w zbiorze.

Tak przygotowane dane umożliwiły analizę wydajności algorytmów dla różnych liczebności i układów punktów.

5.2 Analiza wyników testów wydajnościowych

Dla każdego ze zbiorów wykonano pomiary czasu działania obu algorytmów w celu oceny ich wydajności przy zwiększającej się liczbie punktów. Jednocześnie porównywano zwróconą liczbę punktów tworzących otoczkę, by zweryfikować poprawność działania implementacji. Wyniki przedstawione zostały w tabelach: Tabela 2, Tabela 3, Tabela 4, Tabela 5.

W Zestawie A, dla losowego rozkładu punktów, algorytm Grahama działa zauważalnie szybciej niż algorytm Jarvisa, jest średnio o 32,5% szybszy. Różnica w czasie wykonania rośnie wraz ze wzrostem liczby punktów, ponieważ zwiększa się także liczba punktów na otoczce, co znacząco pogarsza złożoność obliczeniową algorytmu Jarvisa. Oba algorytmy zwróciły taką samą liczbę punktów otoczki wypukłej.

Liczba punktów	Czas Grahama [s]	Czas Jarvisa [s]	Punkty otoczki
100	0.0004	0.0004	8
1000	0.0058	0.0077	16
5000	0.013	0.023	22
10000	0.0248	0.055	27
50000	0.1398	0.231	23

Tabela 2: Porównanie wydajności algorytmów dla Zestawu A

W Zestawie B, w którym dla każdego zbioru wszystkie punkty należą do otoczki wypukłej, algorytm Grahama był średnio aż o 98% szybszy. Algorytm Jarvisa osiąga teoretyczną złożoność rzędu $O(n^2)$, co znajduje potwierdzenie w przeprowadzonych testach, podczas gdy algorytm Grahama utrzymuję niezmienną złożoność rzędu $O(n \log n)$.

Liczba punktów	Czas Grahama [s]	Czas Jarvisa [s]	Punkty otoczki
10	0.0	0.0	10
100	0.0001	0.002	100
500	0.0008	0.0492	500
1000	0.0018	0.1975	1000
5000	0.0106	5.1506	5000

Tabela 3: Porównanie wydajności algorytmów dla Zestawu B

W Zestawie C algorytm Jarvisa wykazuje się lepszą efektywnością niż algorytm Grahama. Wynika to z faktu, że liczba punktów tworzących otoczkę $k = 8$ jest niewielka w porównaniu do całkowitej liczby punktów n. W efekcie czynnik k występujący w złożoności $O(nk)$ można uznać za stały, co upraszcza ją do postaci $O(n)$. Natomiast algorytm Grahama, którego złożoność wynosi $O(n \log n)$, traci na wydajności przy takich danych, co potwierdzają wyniki testów czasowych.

Liczba punktów	Czas Grahama [s]	Czas Jarvisa [s]	Punkty otoczki
100	0.0002	0.0002	8
1000	0.0022	0.002	8
5000	0.0137	0.0104	8
10000	0.0301	0.0211	8
50000	0.1773	0.1115	8

Tabela 4: Porównanie wydajności algorytmów dla Zestawu C

W Zestawie D różnice w wydajności między algorytmem Grahama a algorytmem Jarvisa są jeszcze bardziej widoczne. Wynika to z faktu, że liczba punktów tworzących otoczkę jest stała $k = 4$, niezależna od liczności całego zbioru. Algorytm Jarvisa osiągu złożoność $O(4n) = O(n)$, która sprawia, że działa on o około 63 % szybciej niż algorytm Grahama

Liczba punktów	Czas Grahama [s]	Czas Jarvisa [s]	Punkty otoczki
100	0.0002	0.0001	4
1000	0.0026	0.0011	4
5000	0.0169	0.0056	4
10000	0.0345	0.0111	4
50000	0.2102	0.0587	4

Tabela 5: Porównanie wydajności algorytmów dla Zestawu D

5. Wnioski

Przeprowadzone testy i wizualizacje potwierdziły poprawność działania algorytmów Grahama i Jarvisa — w każdym przypadku wyznaczały one minimalną otoczkę wypukłą obejmującą wszystkie punkty danego zbioru.

Można było zauważyć, że wyniki czasowe działania algorytmów zmieniają się znacząco w zależności od rozkładu punktów. W przypadku równomiernego rozmieszczenia (Zestaw A) lepsze rezultaty osiągał algorytm Grahama, którego złożoność $O(n \log n)$ sprzyja pracy z dużą liczbą danych. Podobna sytuacja wystąpiła w Zestawie B, gdzie wszystkie punkty znajdowały się na otoczce — wówczas czas działania Jarvisa gwałtownie wzrastał (praktycznie $O(n^2)$), podczas gdy Graham zachowywał stałą, znacznie lepszą wydajność.
Natomiast algorytm Jarvisa, działający w czasie $O(nk)$, był bardziej efektywny przy niewielkiej liczbie punktów należących do otoczki (Zestawy C i D), gdzie jego działanie można przybliżyć do złożoności $O(n)$.

Na podstawie przeprowadzonych analiz można stwierdzić, że dobór algorytmu do wyznaczania otoczki wypukłej powinien zależeć od specyfiki i rozmieszczenia punktów w danym zbiorze.